5.6 Multidimensional Data Sets

Lists, tuples, and strings in Python are one-dimensional. We use a single index to access each element of the sequence. Many computer applications involve multidimensional data sets. For example, computer graphics are often modeled in either two or three dimensions. Geographic information may be naturally represented in two dimensions, medical imaging may provide three-dimensional scans of a patient, and a company's valuation is often based upon a high number of independent financial measures that can be modeled as multidimensional data. A two-dimensional array is sometimes also called a ***matrix***. We may use two indices, say i and j, to refer to the cells in the matrix. The first index usually refers to a row number and the second to a column number, and these are traditionally zero-indexed in computer science. Figure 5.22 illustrates a two-dimensional data set with integer values. This data might, for example, represent the number of stores in various regions of Manhattan.

	0	1	2	3	4	5	6	7	8	9
0	22	18	709	5	33	10	4	56	82	440
1	45	32	830	120	750	660	13	77	20	105
2	4	880	45	66	61	28	650	7	510	67
3	940	12	36	3	20	100	306	590	0	500
4	50	65	42	49	88	25	70	126	83	288
5	398	233	5	83	59	232	49	8	365	90
6	33	58	632	87	94	5	59	204	120	829
7	62	394	3	4	102	140	183	390	16	26

Figure 5.22: Illustration of a two-dimensional integer data set, which has 8 rows and 10 columns. The rows and columns are zero-indexed. If this data set were named stores, the value of stores[3][5] is 100 and the value of stores[6][2] is 632.

A common representation for a two-dimensional data set in Python is as a list of lists. In particular, we can represent a two-dimensional array as a list of rows, with each row itself being a list of values. For example, the two-dimensional data

$$\begin{array}{ccccc} 22 & 18 & 709 & 5 & 33 \\ 45 & 32 & 830 & 120 & 750 \\ 4 & 880 & 45 & 66 & 61 \end{array}$$

might be stored in Python as follows.

```
data = [ [22, 18, 709, 5, 33], [45, 32, 830, 120, 750], [4, 880, 45, 66, 61] ]
```

An advantage of this representation is that we can naturally use a syntax such as data[1][3] to represent the value that has row index 1 and column index 3, as data[1], the second entry in the outer list, is itself a list, and thus indexable.

Constructing a Multidimensional List

To quickly initialize a one-dimensional list, we generally rely on a syntax such as `data = [0] * n` to create a list of n zeros. On page 189, we emphasized that from a technical perspective, this creates a list of length n with all entries referencing the same integer instance, but that there was no meaningful consequence of such aliasing because of the immutability of the `int` class in Python.

We have to be considerably more careful when creating a list of lists. If our goal were to create the equivalent of a two-dimensional list of integers, with r rows and c columns, and to initialize all values to zero, a flawed approach might be to try the command

```
data = ([0] * c) * r                # Warning: this is a mistake
```

While `([0] * c)` is indeed a list of c zeros, multiplying that list by r unfortunately creates a single list with length $r \cdot c$, just as `[2,4,6] * 2` results in list `[2, 4, 6, 2, 4, 6]`.

A better, yet still flawed attempt is to make a list that contains the list of c zeros as its only element, and then to multiply that list by r. That is, we could try the command

```
data = [ [0] * c ] * r              # Warning: still a mistake
```

This is much closer, as we actually do have a structure that is formally a list of lists. The problem is that all r entries of the list known as `data` are references to the same instance of a list of c zeros. Figure 5.23 provides a portrayal of such aliasing.

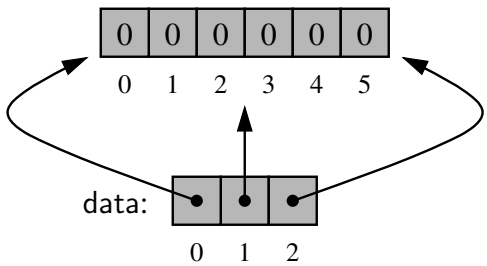


Figure 5.23: A flawed representation of a 3×6 data set as a list of lists, created with the command `data = [ [0] * 6 ] * 3`. (For simplicity, we overlook the fact that the values in the secondary list are referential.)

This is truly a problem. Setting an entry such as `data[2][0] = 100` would change the first entry of the secondary list to reference a new value, 100. Yet that cell of the secondary list also represents the value `data[0][0]`, because "row" `data[0]` and "row" `data[2]` refer to the same secondary list.

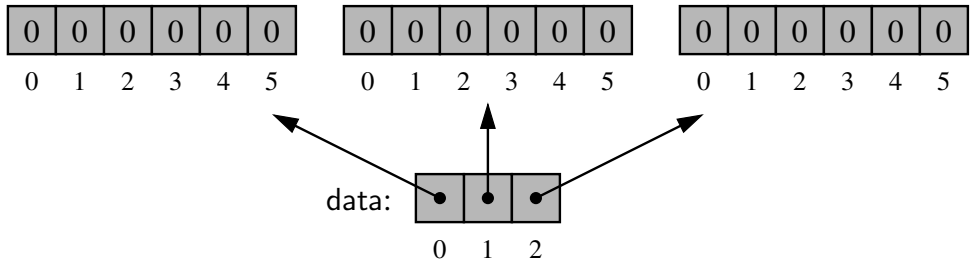


Figure 5.24: A valid representation of a 3×6 data set as a list of lists. (For simplicity, we overlook the fact that the values in the secondary lists are referential.)

To properly initialize a two-dimensional list, we must ensure that each cell of the primary list refers to an *independent* instance of a secondary list. This can be accomplished through the use of Python's list comprehension syntax.

```
data = [ [0] * c for j in range(r) ]
```

This command produces a valid configuration, similar to the one shown in Figure 5.24. By using list comprehension, the expression `[0] * c` is reevaluated for each pass of the embedded for loop. Therefore, we get r distinct secondary lists, as desired. (We note that the variable j in that command is irrelevant; we simply need a for loop that iterates r times.)

Two-Dimensional Arrays and Positional Games

Many computer games, be they strategy games, simulation games, or first-person conflict games, involve objects that reside in a two-dimensional space. Software for such ***positional games*** need a way of representing such a two-dimensional "board," and in Python the list of lists is a natural choice.

Tic-Tac-Toe

As most school children know, ***Tic-Tac-Toe*** is a game played in a three-by-three board. Two players—X and O—alternate in placing their respective marks in the cells of this board, starting with player X. If either player succeeds in getting three of his or her marks in a row, column, or diagonal, then that player wins.

This is admittedly not a sophisticated positional game, and it's not even that much fun to play, since a good player O can always force a tie. Tic-Tac-Toe's saving grace is that it is a nice, simple example showing how two-dimensional arrays can be used for positional games. Software for more sophisticated positional games, such as checkers, chess, or the popular simulation games, are all based on the same approach we illustrate here for using a two-dimensional array for Tic-Tac-Toe.

Our representation of a 3×3 board will be a list of lists of characters, with `'X'` or `'O'` designating a player's move, or `' '` designating an empty space. For example, the board configuration

O	X	O
	X	
	O	X

will be stored internally as

```
[ ['O', 'X', 'O'], [' ', 'X', ' '], [' ', 'O', 'X'] ]
```

We develop a complete Python class for maintaining a Tic-Tac-Toe board for two players. That class will keep track of the moves and report a winner, but it does not perform any strategy or allow someone to play Tic-Tac-Toe against the computer. The details of such a program are beyond the scope of this chapter, but it might nonetheless make a good course project (see Exercise P-8.68).

Before presenting the implementation of the class, we demonstrate its public interface with a simple test in Code Fragment 5.12.

```
1  game = TicTacToe( )
2  # X moves:            # O moves:
3  game.mark(1, 1);      game.mark(0, 2)
4  game.mark(2, 2);      game.mark(0, 0)
5  game.mark(0, 1);      game.mark(2, 1)
6  game.mark(1, 2);      game.mark(1, 0)
7  game.mark(2, 0)
8
9  print(game)
10 winner = game.winner( )
11 if winner is None:
12   print('Tie')
13 else:
14   print(winner, 'wins')
```

Code Fragment 5.12: A simple test for our Tic-Tac-Toe class.

The basic operations are that a new game instance represents an empty board, that the mark(i,j) method adds a mark at the given position for the current player (with the software managing the alternating of turns), and that the game board can be printed and the winner determined. The complete source code for the TicTacToe class is given in Code Fragment 5.13. Our mark method performs error checking to make sure that valid indices are sent, that the position is not already occupied, and that no further moves are made after someone wins the game.

```
class TicTacToe:
  """Management of a Tic-Tac-Toe game (does not do strategy)."""

  def __init__(self):
    """Start a new game."""
    self._board = [ [' '] * 3 for j in range(3) ]
    self._player = 'X'

  def mark(self, i, j):
    """Put an X or O mark at position (i,j) for next player's turn."""
    if not (0 <= i <= 2 and 0 <= j <= 2):
      raise ValueError('Invalid board position')
    if self._board[i][j] != ' ':
      raise ValueError('Board position occupied')
    if self.winner( ) is not None:
      raise ValueError('Game is already complete')
    self._board[i][j] = self._player
    if self._player == 'X':
      self._player = 'O'
    else:
      self._player = 'X'

  def _is_win(self, mark):
    """Check whether the board configuration is a win for the given player."""
    board = self._board                                  # local variable for shorthand
    return (mark == board[0][0] == board[0][1] == board[0][2] or     # row 0
            mark == board[1][0] == board[1][1] == board[1][2] or     # row 1
            mark == board[2][0] == board[2][1] == board[2][2] or     # row 2
            mark == board[0][0] == board[1][0] == board[2][0] or     # column 0
            mark == board[0][1] == board[1][1] == board[2][1] or     # column 1
            mark == board[0][2] == board[1][2] == board[2][2] or     # column 2
            mark == board[0][0] == board[1][1] == board[2][2] or     # diagonal
            mark == board[0][2] == board[1][1] == board[2][0])       # rev diag

  def winner(self):
    """Return mark of winning player, or None to indicate a tie."""
    for mark in 'XO':
      if self._is_win(mark):
        return mark
    return None

  def __str__(self):
    """Return string representation of current game board."""
    rows = ['|'.join(self._board[r]) for r in range(3)]
    return '\n-----\n'.join(rows)
```

Code Fragment 5.13: A complete Python class for managing a Tic-Tac-Toe game.